Complete Django Setup & Development Guide

From Environment Troubleshooting to Building Views and Editing in VS Code

Django 6.x | Windows Environment | Visual Studio Code | Step-by-Step Architecture

Overview of What Was Solved

You encountered `'django-admin' is not recognized as an internal or external command`. We diagnosed that Django 6.1 was installed in Python, identified why Windows could not find the executable, added Python's `Scripts` directory to the Windows PATH, initialized project `mysite`, configured a modern app layout with custom views, and connected VS Code.

1 Understanding the PATH Issue & Diagnosis

When you execute a command in Windows Command Prompt, the operating system does not scan your entire hard drive. It references the **PATH Environment Variable**, an ordered list of directories containing executable files (`.exe`, `.bat`, `.cmd`).

Diagnostic Commands

```
# 1. Verify Django is installed and check its version python -m django --version # Output: 6.1 (Confirms installation exists inside Python) # 2. Locate the exact directory where django-admin.exe is installed python -c "import sysconfig; print(sysconfig.get_path('scripts'))" # Output: C:\Users\Advik05\AppData\Local\Programs\Python\PythonXX\Scripts
```

Why python -m django works when django-admin fails

`python -m django` invokes Python directly to load and run the module `django` inside its internal site-packages. It does not depend on the Windows terminal knowing where `django-admin.exe` lives.

2 Permanent Windows PATH Setup

To ensure `django-admin` works seamlessly in any terminal session without specifying full paths:

Step	Action	Purpose
1. Copy Path	Run `python -c "import sysconfig; print(sysconfig.get_path('scripts'))"`	Pinpoints exact location of `django-admin.exe`
2. Open System Properties	Press `Win` → type *"Environment Variables"* → Click *"Edit system environment variables"*	Opens Windows System Properties modal
3. Edit Path	Click *Environment Variables...* → Select **Path** under User Variables → Click *Edit...*	Accesses individual executable search directories
4. Append & Save	Click *New* → Paste Scripts folder path → Click *OK* on all windows	Permanently registers directory into Windows lookup
5. Restart Terminal	Close open Command Prompt windows & open a fresh terminal	Reloads system environment registry state

3 Initializing Project Structure

Once `django-admin` is operational, initialize your site and verify internal files:

```
# Create new Django project django-admin startproject mysite # Move inside the project root cd mysite # Run database migrations for built-in auth, admin, and sessions python manage.py migrate
```

Project Directory Hierarchy

```
mysite/ <-- Project Root (Contains manage.py) |-- manage.py <-- Command-line utility for local administrative tasks |-- db.sqlite3 <-- Default lightweight database `-- mysite/ <-- Inner project configuration directory |-- __init__.py <-- Tells Python this directory is a package |-- settings.py <-- All global configurations, installed apps, DB |-- urls.py <-- Main routing / top-level URL declarations |-- asgi.py <-- Asynchronous gateway interface `-- wsgi.py <-- Web Server Gateway Interface (for deployment)
```

4 Creating an App & Adding First View

In Django terminology, a **project** contains the overall website configuration, while an **app** is a modular component (e.g., blog, store, authentication system).

Step A: Generate the App

```
python manage.py startapp main
```

Step B: Register App in `mysite/settings.py`

```
INSTALLED_APPS = [ 'django.contrib.admin', 'django.contrib.auth', 'django.contrib.contenttypes', 'django.contrib.sessions', 'django.contrib.messages', 'django.contrib.staticfiles', 'main', # <-- Newly registered application ]
```

Step C: Create View Logic in `main/views.py`

```
from django.http import HttpResponse def home(request): return HttpResponse("<h1>Welcome to my Django site!</h1><p>Running smoothly via VS Code.</p>")
```

Step D: Map View to Root URL in `mysite/urls.py`

```
from django.contrib import admin from django.urls import path from main.views import home # Import the view function urlpatterns = [ path('admin/', admin.site.urls), path('', home, name='home'), # Maps root domain ('/') to home() view ]
```

5 Visual Studio Code Integration

Working inside Visual Studio Code consolidates code editing, file navigation, and server debugging into a unified workspace.

Action	Command / Shortcut	Details
Launch directly from CMD	`code .`	Opens the current directory (`mysite`) directly inside VS Code
Integrated Terminal	`Ctrl` + `~`	

Action	Command / Shortcut	Details
		Opens an integrated terminal inside VS Code already inside `mysite/`
Select Python Interpreter	`Ctrl` + `Shift` + `P` → *Python: Select Interpreter*	Ensures VS Code uses the exact Python 6.1 environment
Recommended Extensions	Python (by Microsoft) Django (by Baptiste Darthenay)	Provides syntax highlighting, auto-imports, and HTML template snippets

Starting Local Server inside VS Code Terminal

```
python manage.py runserver
```

Open `http://127.0.0.1:8000/` in your browser to inspect your application live. Press `Ctrl` + `C` in the terminal whenever you want to halt the server.

6 Essential Django Commands Cheat Sheet

Task	Command
Create new project	`django-admin startproject <project_name>`
Create new app	`python manage.py startapp <app_name>`
Apply database migrations	`python manage.py migrate`
Create migration files for models	`python manage.py makemigrations`
Create admin superuser	`python manage.py createsuperuser`
Launch local development server	`python manage.py runserver`
Open interactive Django Python shell	`python manage.py shell`
Check project for potential errors	`python manage.py check`